

Dynamic Polynomial Regressions: An approach to Function Discovery

Gavin Simpson

May 31, 2026

1 Overview

Function discovery is all about finding an equation to best represent the relationship between variables. Today we will discuss a simple 2 Dimensional dataset (X, Y) .

1.1 The Approach

We are not going to be using a symbolic regression today, instead we'll be advancing to a hybrid Polynomial/Power Law regression.

This is simply done by making our Regression function $(\hat{y}_i)$ more advanced instead of a linear $y = mx + c$.

But with this approach, we wont be able to use advanced functions that a symbolic regression could use such as $\sin x$ or $\ln x$.

With this approach, we'll be using the same MSE and performing the **gradient descent** to find the most optimal parameters. The main difficulty here is going to be the partial derivatives (especially when it turns dynamic).

$$\text{MSE}(\text{parameters}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Lets start with a simply polynomial regression to warm up

2 Fixed Degree Polynomial Regressions

The polynomial regression works by making the regression function $(\hat{y}_i)$ a polynomial. Some Examples:

$$\begin{aligned}\hat{y} &= ax^2 + bx + c \\ \hat{y} &= ax^3 + bx^2 + cx + d\end{aligned}$$

With our Polynomial regressions, the procedure will be almost identical to the linear regression.

For example, the steps for a 3 Degree polynomial regression

1. The new loss function

$$\text{MSE}(a,b,c,d) = \frac{1}{n} \sum_{i=1}^n (ax_i^3 + bx_i^2 + cx_i + d - y_i)^2$$

2. The Partial Derivatives

$$\frac{\partial}{\partial a} \Big|_{a,b,c,d} = \frac{2}{n} \sum_{i=1}^n (ax_i^3 + bx_i^2 + cx_i + d - y_i)^1 \cdot x_i^3$$

$$\frac{\partial}{\partial b} \Big|_{a,b,c,d} = \frac{2}{n} \sum_{i=1}^n (ax_i^3 + bx_i^2 + cx_i + d - y_i)^1 \cdot x_i^2$$

$$\frac{\partial}{\partial c} \Big|_{a,b,c,d} = \frac{2}{n} \sum_{i=1}^n (ax_i^3 + bx_i^2 + cx_i + d - y_i)^1 \cdot x_i$$

$$\frac{\partial}{\partial d} \Big|_{a,b,c,d} = \frac{2}{n} \sum_{i=1}^n (ax_i^3 + bx_i^2 + cx_i + d - y_i)^1 \cdot 1$$

3. The Iterative Steps (α = a suitable rate)

$$a = a - \alpha \cdot \frac{\partial}{\partial a} \Big|_{a,b,c,d}$$

$$b = b - \alpha \cdot \frac{\partial}{\partial b} \Big|_{a,b,c,d}$$

$$c = c - \alpha \cdot \frac{\partial}{\partial c} \Big|_{a,b,c,d}$$

$$d = d - \alpha \cdot \frac{\partial}{\partial d} \Big|_{a,b,c,d}$$

The Polynomial regression will find the best cubic function by finding the parameters (a,b,c,d) that result in the lowest MSE.

3 Nth Degree Polynomial with Fixed Terms

What if we take this a step higher? Right now with our polynomial regression, we only tweak the coefficients of our regression function. But what if we also tweak the exponents too....

$$\hat{y} = c_1x^{e_1} + c_2x^{e_2} + c_3x^{e_3}$$

where

- c_1, c_2, c_3 are the coefficients that will be discovered
- e_1, e_2, e_3 are the exponents that will be discovered

in this case our loss function would look a bit different....

$$\text{MSE}(c_1, c_2, c_3, e_1, e_2, e_3) = \frac{1}{n} \sum_{i=1}^n (c_1x^{e_1} + c_2x^{e_2} + c_3x^{e_3} - y_i)^2$$

This would also mean that we would have different partial derivatives. But we can generalize them and find just two - one for the coefficients and one for the exponents using common calculus methods.

$$\frac{\partial}{\partial c_n} = \frac{2}{n} \sum_{i=1}^n (c_n x_i^{e_n} \dots - y_i)^1 \cdot x_i^{e_n}$$

$$\frac{\partial}{\partial e_n} = \frac{2}{n} \sum_{i=1}^n (c_n x_i^{e_n} \dots - y_i)^1 \cdot c_n \cdot [(x_i^{e_n}) \ln(x_i)]$$

(for the partial derivative of the exponent - use implicit differentiation)

The algorithm proceeds as usual

The Iteration Steps:

For the Nth coefficient

$$c_n = c_n - \alpha \cdot \frac{\partial}{\partial c_n} \Big|_{c_n \dots e_n \dots}$$

For the Nth exponent

$$e_n = e_n - \alpha \cdot \frac{\partial}{\partial e_n} \Big|_{c_n \dots e_n \dots}$$

3.1 Safety Measures

There are two new steps we need to take in order to prevent the algorithm from exploding.

3.1.1 Normalization

Normalising your dataset simply means squashing all of our values into **0 and 1**. EG: The whole Sin X function that goes from 0 to 10 (X), is now squashed into 0 and 1 VIA a normalisation function

$$x_{norm} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Why should we Normalise? Our Partial derivative for the Exponents, include a $\ln(x)$, If the Values of X are big, we'll get massive gradients which can shoot up the Exponents. Which can result in overflow errors (values too big for variables).

Additionally we will need to drop all values that are **0** as $\ln 0 \rightarrow -\infty$

3.1.2 Exponent Clamping

We can also Clamp our exponents within a range (between p and q). This is another safeguard against exponent explosions.

In python we can use the **Numpy.clip** function

```
RegressionTerms[TermIndex][1] = np.clip(RegressionTerms[TermIndex][1],
min_exp, max_exp)
```

3.2 Dual Learning Rates

The exponents are very sensitive, whereas with the coefficients, we want traction, hence the learning rates wouldn't make sense if they were the same, hence we can use dual learning rates

$$\alpha_c = 0.01$$

$$\alpha_e = 1e^{-6}$$

The Iteration Steps:

$$c_n = c_n - \alpha_c \cdot \frac{\partial}{\partial c_n}$$

$$e_n = e_n - \alpha_e \cdot \frac{\partial}{\partial e_n}$$

4 Nth Degree Polynomial with R Terms

Previously, we discussed the case with fixed terms, in the below function, there are only 3 terms

$$\hat{y} = c_1x^{e_1} + c_2x^{e_2} + c_3x^{e_3}$$

What if we can change this, make it N terms....

We can transform this algorithm and make it *genetic* with **term creation** and **term pruning**. The algorithm will start with a set of default terms and new terms will start to get created from stronger terms, then after a warm up period, weaker terms (that don't contribute to the function) will start to get pruned out.

4.1 Term Creation (Feature Augmentation)

When creating terms - there are many ways to go about this

- Creating new terms from stronger terms with larger coefficients by duplication and a small varying in the exponent - though you must avoid tripping the pruning....
- Creating new terms from stronger terms with lower gradients
- Creating new terms randomly: This adds diversity to the algorithm and it can also add a symbolic regression flavor. (as well as an exit incase the algorithm has hit a local minima)

4.2 Term Pruning

An approach to remove bloat - over iterations of term creation, the regression function may become 'bloated' with terms that contribute little to no value, this will lead to extra compute (derivatives) for no reason, hence Pruning is used to filter out these terms.

4.2.1 L1 Regularization (AKA Lasso)

L1 Regularization is where we penalize terms on how much impact they have on the overall regression. Terms with larger coefficients contribute more to the regression and terms with smaller coefficients contribute less (as the whole term is close to 0).

L1 Regularisation works by pushing weaker terms towards 0, the 'aggressiveness' - how hard it pushes, depends on λ (L1 Regularization Coefficient), Different Values of λ

Value	Effect
0.0001	barely deletes anything
0.001	light pruning
0.01	strong pruning
0.1	brutal pruning — will kill most terms

Table 1: Choosing λ (Lambda) Values

Specifically, λ is multiplied to the gradient in the opposite direction. (*Note: Only for the partial derivatives of the coefficient - for obvious reasons*)

$$\text{Total Gradient for } C_n = \left(\frac{\partial}{\partial C_n} \right) + \lambda_{L1} \cdot \text{sign}(x_i)$$

4.2.2 Pruning

Every 'set' times, we clear all the terms that have their co-efficients close to the **threshold** (which could be about **1e-5**).

Additionally we get rid of Repeated terms that accumulate at the end. This is typically a symptom of them hitting the upper exponent bound as part of the Exponent Clamping. We have a tolerance between how similar the coefficient **and** exponent must be, in order to eliminate one of them.

4.2.3 Term Merging

Additionally we can also merge terms that have exponents that are very close (but with different coefficients). This too prevents unnecessary bloat.

5 Further Resources

Checkout some implementations of these function discovery methods that I've done

- Github Repository
- Polynomial Regression - Demo
- Power Law Regression - L1 Regularisation and Feature Augmentation (Largest Coefficient) - Demo
- Power Law Regression - L1 Regularisation and Feature Augmentation (Lowest Exponent Gradient) - Demo

Thank you for reading this Document, I hope it has helped in some small way (not small enough to get pruned hopefully haha...)